Git workflows & CI runners

An illustrated comparison for MyScoutee Backend. English learning guide, based on the two shared conversations and primary documentation. [1-2]

A branching strategy organizes **how code changes meet**. A pipeline describes **what work to run**. A runner provides **where that work executes**. You can change one without automatically changing the others.

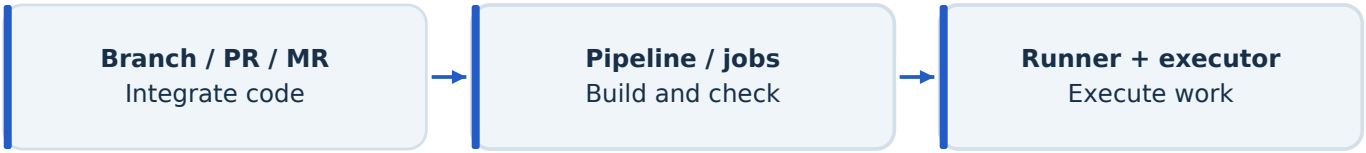


Figure 1. Three decisions in one delivery system. Arrows show the conceptual work flow, not Git merge edges.

Question	Mechanism	Read
When and where do changes merge?	Gitflow, GitHub Flow, GitLab Flow, trunk-based development	2-4
How do teams contribute and release?	Feature/release/environment branches, forks, GitOps, Ship/Show/Ask	5
Which projects and machines can run a job?	Runner scope, tags and executors	6
How many jobs exist, and how many run?	parallel, concurrent, limit, request_concurrency	7-8
How does this relate to this repository?	MyScoutee ownership and examples	9
Where can I verify the details?	Sources and reading paths	10

Vocabulary. PR means pull request; MR means merge request. Both propose integrating changes. The diagrams use **main** generically; this repository uses **master**.

Document status. Educational guidance, not a new repository policy or a report of live runner settings. Examples are illustrative; no runner or deployment configuration is changed by this guide.

Four integration models

Compare the integration point and release process before choosing branch names. [3-7]

Model	Integration and release	Trade-off / useful context
Gitflow	Features merge into develop. A release branch stabilizes a version; main records releases.	Explicit release preparation, with more branches and merge paths. Useful for scheduled, versioned releases.
GitHub Flow	Short branches propose changes to the default branch through PRs.	Simple collaboration structure. Release and deployment gates still need to be defined.
GitLab Flow	Feature branches merge into main; environment or stable-version branches may extend the flow.	Models release constraints explicitly, with extra synchronization when extra branches are used.
Trunk-based	Small changes integrate into one trunk frequently; short-lived branches and review are compatible.	Early integration feedback; needs fast checks, small slices and prompt review.

The same feature, four routes

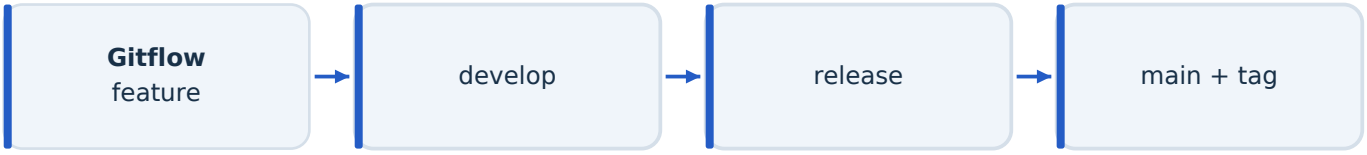


Figure 2a. Release fixes also return to develop; see the full diagram on page 3.

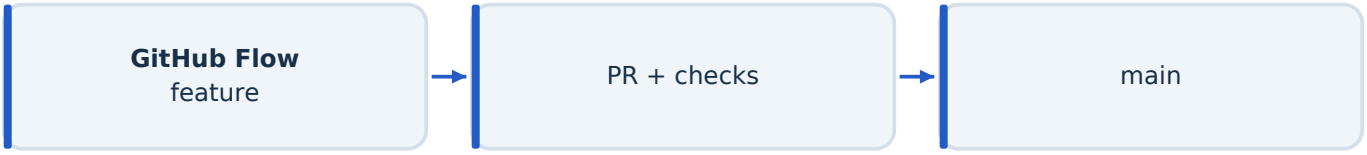


Figure 2b. Deployment is a separately defined step after, or around, integration.



Figure 2c. One environment-branch variant; production is not mandatory in every variant.

Trunk-based: several small changes enter main throughout development, instead of waiting for the entire feature. GitHub-style PRs can implement this when they remain short-lived.

Separate development from release preparation

Original Gitflow roles, drawn as conceptual merge paths rather than an exact commit graph. [3]

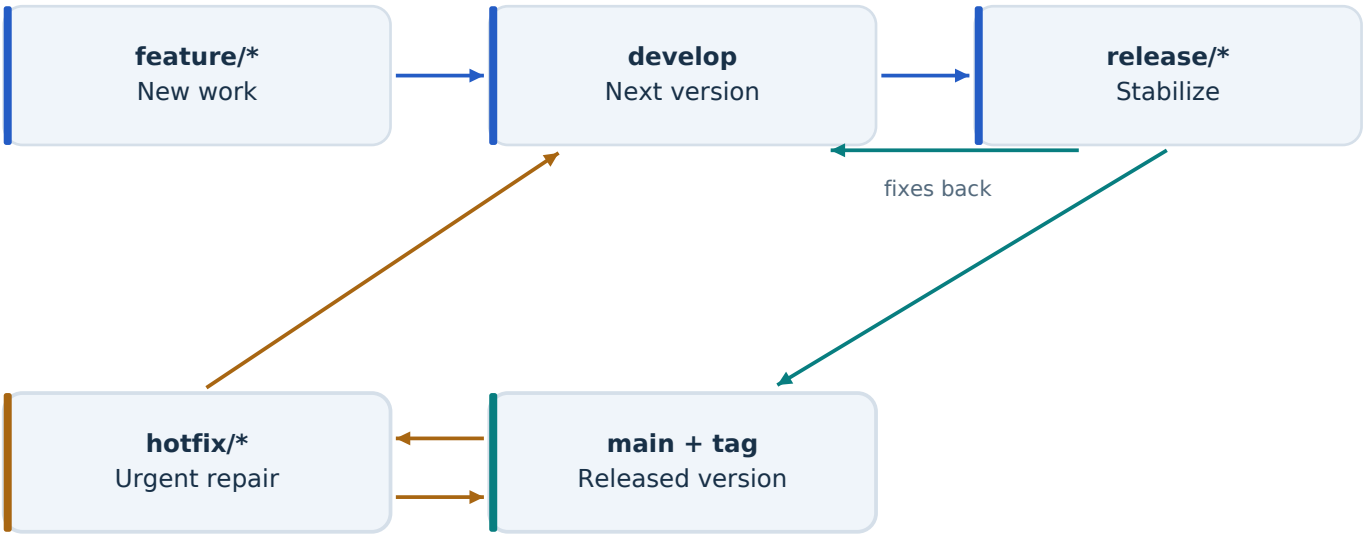


Figure 3. Blue: feature delivery. Teal: release completion and fixes returned to development. Gold: hotfix branches from main and returns to main and develop.

feature/* starts from develop and returns there. **release/*** starts from develop, stabilizes the next version, then merges into main and develop. Tag the release on main.

hotfix/* starts from the released main line. Merge the repair into main and back into development; if a release branch is active, incorporate it there so the next release retains the fix.

Benefit. Release stabilization can proceed while development of the next version continues.

Cost. More long-lived lines create more synchronization work and opportunities to miss a back-merge.

When to consider it. A deliberate release train with version support requirements. The original author later recommended simpler flows for continuously delivered web applications.

GitHub Flow and trunk-based development

These can overlap: one describes a collaboration flow; the other emphasizes frequent integration. [4, 6-7]

GitHub Flow: propose, review, merge

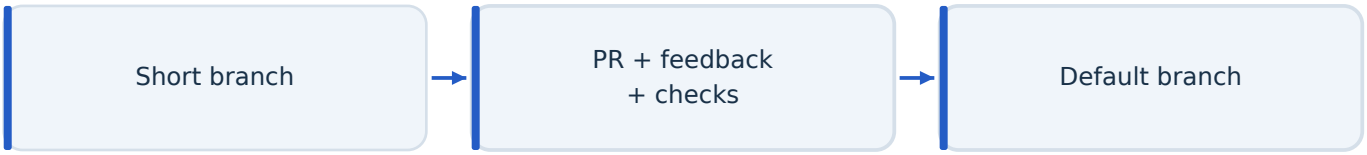


Figure 4. The branch isolates a change while collaborators review it.

Create a focused branch, commit changes, open a pull request, address feedback, then merge and remove the finished branch. Required reviews and checks come from repository configuration. A feature branch that stays open for weeks may follow this shape while accumulating substantial integration risk.

Trunk-based: integrate small slices frequently

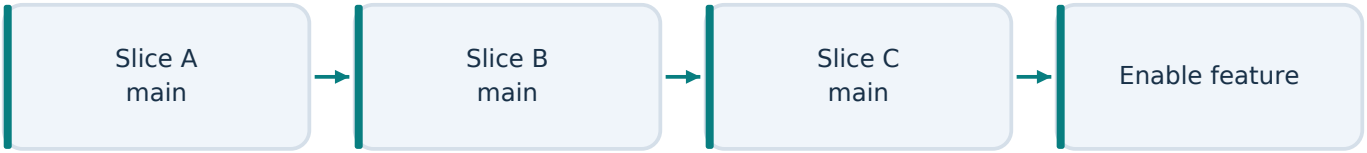


Figure 5. An illustrative feature-flag sequence. Integration and user-visible activation need not happen together.

Keep a single shared development line. Direct commits or short-lived reviewed branches can both work. Review waiting time counts toward branch lifetime. Break large features into compatible slices; feature flags or branch-by-abstraction can keep incomplete behavior inactive.

Dimension	GitHub Flow	Trunk-based emphasis
Review	PR discussion before merge	Review that keeps integration frequent
Change size	Focused change	Small, independently safe slice
Key failure mode	Long PR queue / branch drift	Slow or unreliable feedback on trunk

Practical interpretation. The two labels are compatible when PRs integrate promptly. Neither label alone proves that production deployment is automatic or that tests are sufficient.

GitLab Flow and the patterns you combine

The ten names in the conversation are not ten mutually exclusive choices. [5, 8-10]

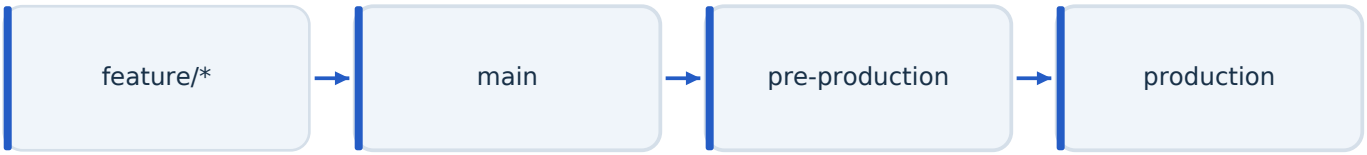


Figure 6. An environment-branch version of GitLab Flow. Promote fixes forward through the defined path.

GitLab Flow combines feature-driven integration with issue tracking and delivery constraints. Environment branches track promotion; stable-version branches support maintained versions. Choose the extra branches for an actual release need. GitLab Runner does not require GitLab Flow.

Pattern	What it changes	How it combines / trade-off
Feature Branch Workflow	A separate branch holds a change.	Building block of several flows; long branch lifetime raises integration cost.
Release Branching	A branch stabilizes or maintains a version.	Can supplement trunk-based development or other flows; fixes must reach every relevant line.
Environment Branching	Branches represent promotion stages.	Can implement a GitLab Flow variant; source branches and runtime environments can drift.
Forking Workflow	Contributions originate in another repository.	Works with PR review; changes repository ownership, not the release cadence.
GitOps	Versioned desired state is pulled and continuously reconciled.	A delivery/operations model. Separate repositories or directories can work; environment branches are optional.
Ship / Show / Ask	Chooses whether review precedes integration.	Ship: integrate without review. Show: integrate, then invite feedback. Ask: discuss before merging.

Example combination. Short feature branches + PR review + a release branch for an older supported version + GitOps deployment can coexist. This is an illustrative composition, not an adopted MyScoutee policy.

Scope, executor and tags answer different questions

A runner is registered in GitLab; its manager executes jobs using an executor. [11-14]

Dimension	Options	Question answered
Scope	Instance / group / project	Which projects can use this runner?
Executor	Shell / Docker / Kubernetes, etc.	How is a job environment created?
Tags	For example: lab-build, lab-docker	Which eligible runner matches the job?
Manager / host	Runner process and its infrastructure	Where is execution capacity supplied?

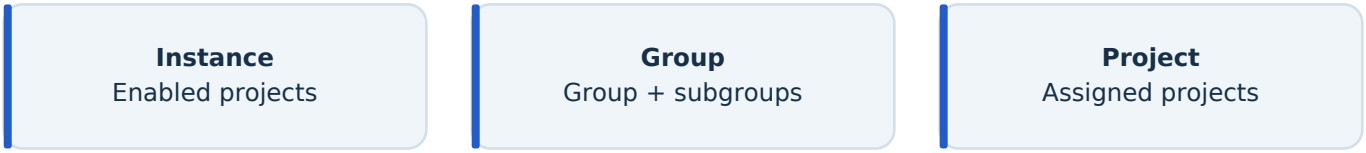


Figure 7. Scope options from broad to narrow. These boxes are alternatives, not a job execution chain.

Instance runners are available across an instance subject to enablement and policy. Group runners serve projects in their group hierarchy. Project runners serve explicitly assigned projects. A project shared with a group grants member access; that invitation does not move it into the group hierarchy or automatically grant the invited group's runner capacity. [11, 13]

Execution isolation is a separate choice

Shell runs commands on the host. **Docker** runs jobs in containers. **Kubernetes** provisions job pods. Their isolation and capabilities depend on configuration: a privileged container or a mounted host Docker socket changes the trust boundary. A new registration on the same host is not, by itself, a strong isolation boundary. [14]

Matching. A runner must have every tag requested by a job. Eligibility, protected-runner rules, availability and capacity also matter. A tag is a selector, not a security sandbox. [12]

Naming. The older term “shared runner” commonly refers to instance scope. “Shared project” describes access sharing. They are different uses of “shared”. [11, 13]

One runner can execute several jobs at once

Separate the amount of work from the capacity that consumes it. [15-16]

Setting	Location	Meaning
parallel	Pipeline YAML	Creates multiple job instances; does not add machines.
concurrent	Manager config.toml	Combined running-job ceiling for this runner process.
limit	One [[runners]] entry	Ceiling for that entry; 0 means no additional entry limit.
request_concurrency	One [[runners]] entry	Concurrent requests for new jobs, not running-job slots.

```
concurrent = 4

[[runners]]
  name = "example-docker"
  executor = "docker"
  limit = 2
  request_concurrency = 1
```

Illustrative configuration fragment; registration credentials and executor-specific settings are omitted. In this example the entry can run at most **two** jobs, even though its manager allows four in total. A request limit of one does not imply only one running job. [15]

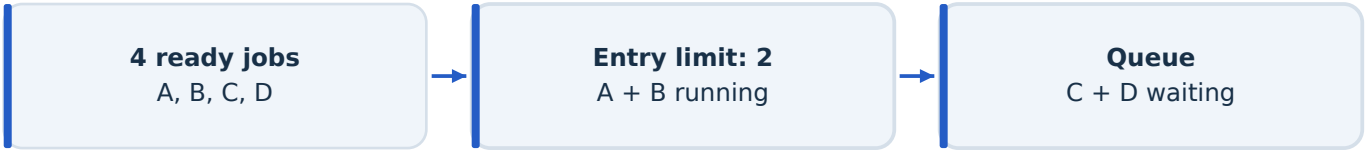


Figure 8. Simplified scheduling example: all jobs match, dependencies are satisfied and no other jobs consume slots.

Computed example. With limit = 0 and concurrent = 4, this entry could occupy all four slots. With two entries limited to three each under one concurrent = 4 manager, their combined ceiling is four, not six.

Multiple managers: two independent four-slot managers have an eight-slot combined ceiling, subject to ready work and infrastructure capacity.

More job instances do not guarantee more speed

Parallel jobs need both capacity and a real division of work. [15-17]

```
tests:
  stage: test
  tags: [example-test]
  parallel: 4
  script:
    - ./run-shard "$CI_NODE_INDEX" "$CI_NODE_TOTAL"
```

Illustrative YAML: `run-shard` is a placeholder for a test splitter you must implement or replace with your test tool's sharding command. GitLab supplies the index and total. Plain `npm test` with parallel: 4 usually repeats the same suite four times unless the test setup consumes those values. [16]

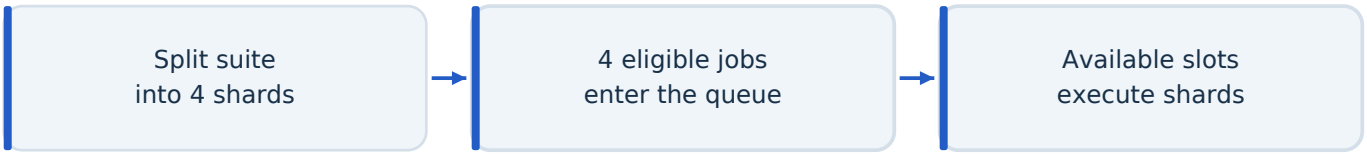


Figure 9. Four created jobs may execute in one wave, two waves, or sequentially.

Need	New runner?
More simultaneous jobs in the same environment	Not necessarily. Check manager/entry limits, ready work and host capacity first.
A different executor or operating system	Usually a separate runner configuration and suitable execution infrastructure.
A GPU or special network access	A dedicated configuration/pool can route suitable jobs; the tag alone supplies no hardware.
Capacity on another host / failure domain	Add execution infrastructure and a manager or autoscaling arrangement.
Deployment separated from untrusted jobs	Use a deliberately separate trust boundary, credentials and admission rules. A second name is insufficient.

Bottlenecks: check eligibility, dependencies, locks and CPU/RAM/I/O. A resource_group deliberately serializes matching jobs within a project. [12, 17]

Apply the distinctions to this repository

Repository evidence and teaching examples are kept distinct. References point to specific source revisions. [18-19]

Verified ownership: `config/ci/pipeline.yml` includes the shared CI pipeline through `LAB_CI_PATH` and `LAB_CI_REF`. The project owns product settings; the shared **myscoutee-ci** repository owns the common pipeline. The effective runner setup requires the actual pinned CI revision and deployed configuration. [18]

Label in shared CI	Documented role	Implication
lab-docker	Small CI validation in an unprivileged container	Select a suitable isolated validation environment.
lab-build	Trusted native builds on the host with Docker access	Host access requires a trusted workload boundary.

These roles come from versioned CI documentation. The labels do not tell you the live concurrent or limit values, the number of hosts, or whether a job is currently running. [19]

Teaching example: an application change

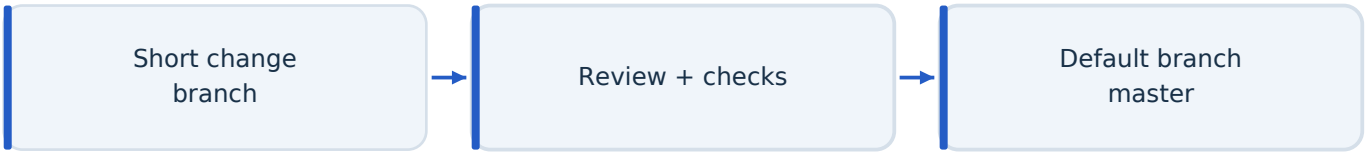


Figure 10a. An illustrative integration route, not a declaration of enforced branch protection.

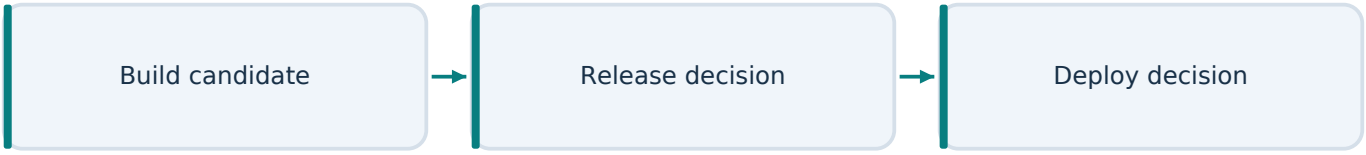


Figure 10b. Distinct decisions. Their ordering and gates belong to the delivery workflow.

For routine changes, short reviewable branches are a simple starting point. Add a maintained release branch only when an older supported version needs independent fixes. Keep “which source is approved”, “which artifact is released” and “where it runs” explicit. This is guidance, not a migration instruction.

Before changing capacity: locate the owning configuration; check isolated work directories, caches and fixtures. Extra registrations do not create CPU or memory or remove shared-state conflicts.

Sources & further reading

Checked 25 October 2026. Original diagrams and worked examples; external sources are paraphrased. Links are clickable.

[1] Shared conversation: workflows and runner scopes

[2] Shared conversation: runner concurrency

[3] Vincent Driessen: A successful Git branching model

[4] GitHub Docs: GitHub flow

[5] GitLab: What is GitLab Flow?

[6] Trunk Based Development: Introduction

[7] Trunk Based Development: Short-Lived Feature Branches

[8] GitHub Docs: About forks

[9] OpenGitOps: Principles

[10] Rouan Wilsenach: Ship / Show / Ask

[11] GitLab Docs: Manage runners (scope)

[12] GitLab Docs: Configuring runners (tags and eligibility)

[13] GitLab Docs: Sharing projects and groups

[14] GitLab Runner: Executors

[15] GitLab Runner: Advanced configuration

[16] GitLab Docs: CI/CD YAML syntax reference, parallel

[17] GitLab Docs: Resource groups

[18] MyScoutee Backend: project CI include (bc9116e35)

[19] MyScoutee CI: runner roles (f5d200257)

How to use this guide

For branching, start with pages 2-5. For runner setup and throughput, start with pages 6-8. Page 9 connects the concepts to MyScoutee without prescribing a new workflow.

Source snapshots describe the cited revisions. Online GitLab documentation can change; check the documentation matching your installed version before applying settings. The two conversations provide the learning questions; primary sources provide the technical reference.